

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]

11

Join Algorithms



Intro to Database Systems
15-445/15-645
Fall 2019

AP

Andy Pavlo
Computer Science
Carnegie Mellon University

WHY DO WE NEED TO JOIN?

We normalize tables in a relational database to avoid unnecessary repetition of information.

We use the join operate to reconstruct the original tuples without any information loss.



JOIN ALGORITHMS

We will focus on combining two tables at a time with **inner equijoin** algorithms.

→ These algorithms can be tweaked to support other joins.

In general, we want the smaller table to always be the left table ("outer table") in the query plan.

JOIN OPERATORS

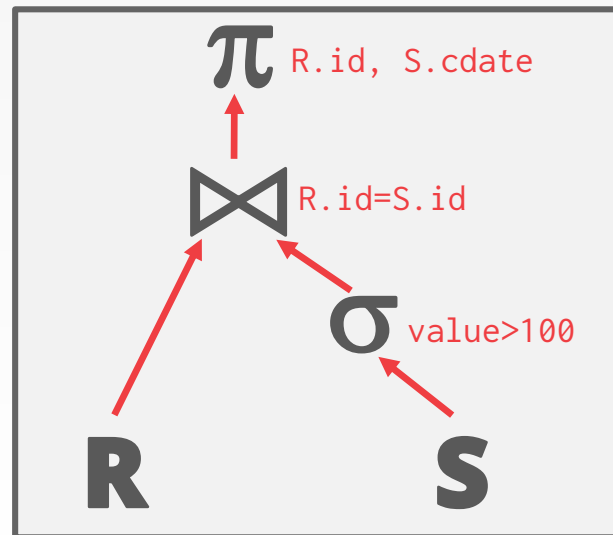
Decision #1: Output

→ What data does the join operator emit to its parent operator in the query plan tree?

Decision #2: Cost Analysis Criteria

→ How do we determine whether one join algorithm is better than another?

```
SELECT R.id, S.cdate  
FROM R JOIN S  
      ON R.id = S.id  
WHERE S.value > 100
```



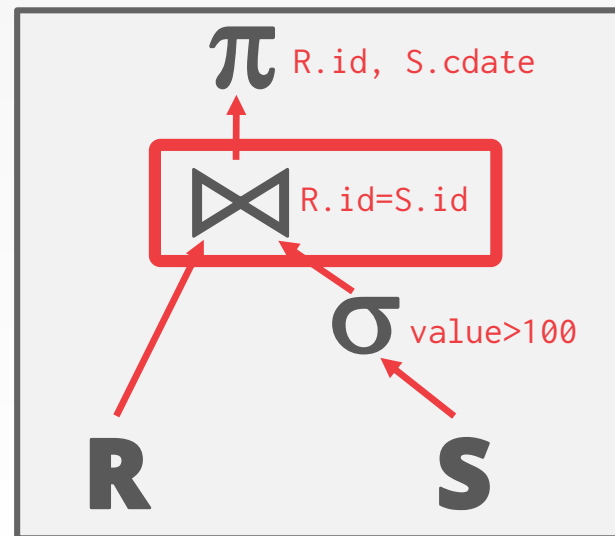
OPERATOR OUTPUT

For a tuple $\mathbf{r} \in \mathbf{R}$ and a tuple $\mathbf{s} \in \mathbf{S}$ that match on join attributes, concatenate $\mathbf{r}$ and $\mathbf{s}$ together into a new tuple.

Contents can vary:

- Depends on processing model
- Depends on storage model
- Depends on the query

```
SELECT R.id, S.cdate  
FROM R JOIN S  
ON R.id = S.id  
WHERE S.value > 100
```



JOIN OPERATOR OUTPUT: DATA

Copy the values for the attributes in outer and inner tuples into a new output tuple.

```
SELECT R.id, S.cdate
FROM R JOIN S
ON R.id = S.id
WHERE S.value > 100
```

R(id,name) **S(id,value,cdatetime)**

id	name		id	value	cdatetime
123	abc		123	1000	10/2/2019
			123	2000	10/2/2019

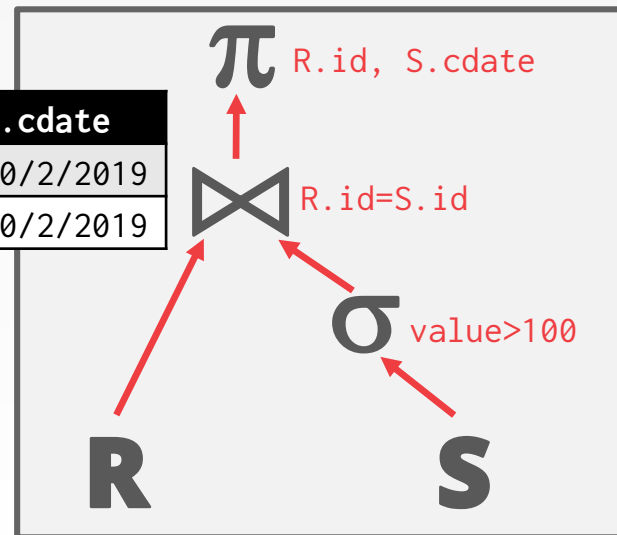
R.id	R.name	S.id	S.value	S.cdate
123	abc	123	1000	10/2/2019
123	abc	123	2000	10/2/2019

JOIN OPERATOR OUTPUT: DATA

Copy the values for the attributes in outer and inner tuples into a new output tuple.

```
SELECT R.id, S.cdate  
FROM R JOIN S  
ON R.id = S.id  
WHERE S.value > 100
```

R.id	R.name	S.id	S.value	S.cdate
123	abc	123	1000	10/2/2019
123	abc	123	2000	10/2/2019

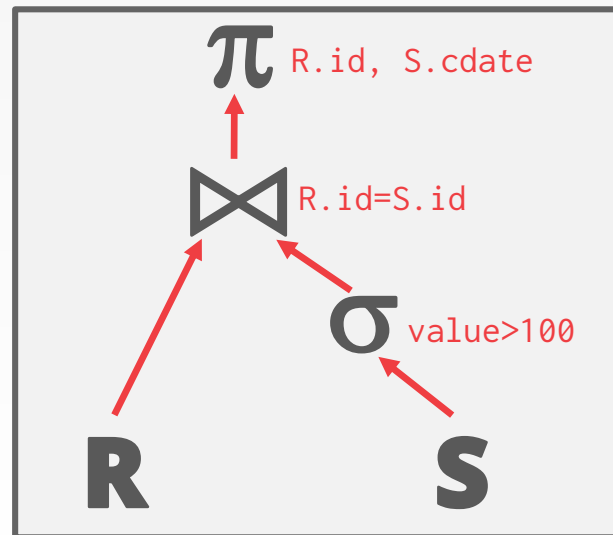


JOIN OPERATOR OUTPUT: DATA

Copy the values for the attributes in outer and inner tuples into a new output tuple.

Subsequent operators in the query plan never need to go back to the base tables to get more data.

```
SELECT R.id, S.cdate  
FROM R JOIN S  
ON R.id = S.id  
WHERE S.value > 100
```



OPERATOR OUTPUT: RECORD IDS

Only copy the joins keys along with the record ids of the matching tuples.

```
SELECT R.id, S.cdate
FROM R JOIN S
ON R.id = S.id
WHERE S.value > 100
```

R(id,name) **S(id,value,cdatetime)**

id	name		id	value	cdatetime
123	abc		123	1000	10/2/2019
			123	2000	10/2/2019



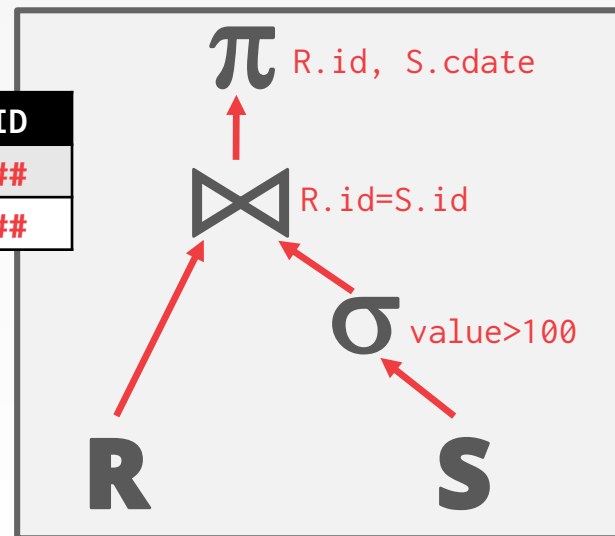
R.id	R.RID	S.id	S.RID
123	R.###	123	S.###
123	R.###	123	S.###

OPERATOR OUTPUT: RECORD IDS

Only copy the joins keys along with the record ids of the matching tuples.

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

R.id	R.RID	S.id	S.RID
123	R.###	123	S.###
123	R.###	123	S.###



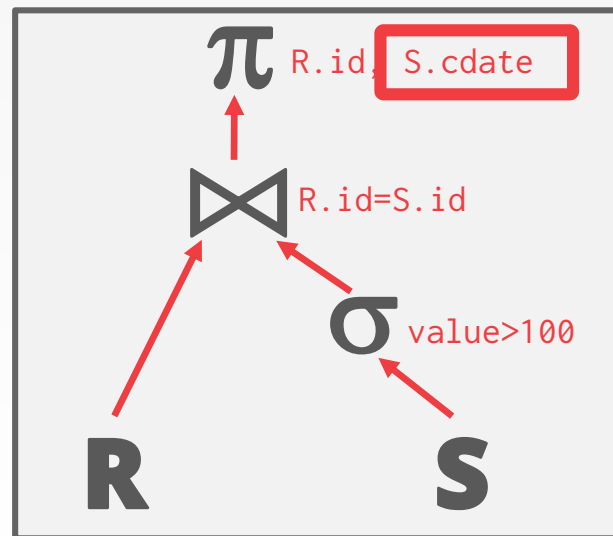
OPERATOR OUTPUT: RECORD IDS

Only copy the joins keys along with the record ids of the matching tuples.

Ideal for column stores because the DBMS does not copy data that is not need for the query.

This is called late materialization.

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```



I/O COST ANALYSIS

Assume:

- M pages in table R , m tuples in R
- N pages in table S , n tuples in S

```
SELECT R.id, S.cdate  
FROM R JOIN S  
      ON R.id = S.id  
WHERE S.value > 100
```

Cost Metric: # of IOs to compute join

We will ignore output costs since that depends on the data and we cannot compute that yet.

JOIN VS CROSS-PRODUCT

$R \bowtie S$ is the most common operation and thus must be carefully optimized.

$R \times S$ followed by a selection is inefficient because the cross-product is large.

There are many algorithms for reducing join cost, but no algorithm works well in all scenarios.

JOIN ALGORITHMS

Nested Loop Join

- Simple / Stupid
- Block
- Index

Sort-Merge Join

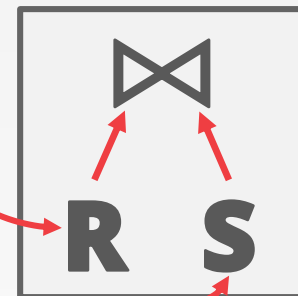
Hash Join



NESTED LOOP JOIN

```

foreach tuple r ∈ R: ← Outer
  foreach tuple s ∈ S: ← Inner
    emit, if r and s match
  
```



R(id,name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

STUPID NESTED LOOP JOIN

Why is this algorithm stupid?

→ For every tuple in **R**, it scans **S** once

Cost: $M + (m \cdot N)$

M pages
 m tuples

R(id, name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

S(id, value, cdate)

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

N pages
 n tuples

STUPID NESTED LOOP JOIN

Example database:

→ Table **R**: $M = 1000$, $m = 100,000$
→ Table **S**: $N = 500$, $n = 40,000$

} *4 KB pages* → *6 MB*

Cost Analysis:

→ $M + (m \cdot N) = 1000 + (100000 \cdot 500) = 50,001,000$ IOs

→ At 0.1 ms/IO, Total time $\approx$ 1.3 hours

What if smaller table (**S**) is used as the outer table?

→ $N + (n \cdot M) = 500 + (40000 \cdot 1000) = 40,000,500$ IOs

→ At 0.1 ms/IO, Total time $\approx$ 1.1 hours

BLOCK NESTED LOOP JOIN

```

foreach block  $B_R \in R$ :
  foreach block  $B_S \in S$ :
    foreach tuple  $r \in B_R$ :
      foreach tuple  $s \in B_S$ :
        emit, if  $r$  and  $s$  match
  
```

$R(id, name)$

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

M pages
 m tuples

$S(id, value, cdate)$

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

N pages
 n tuples

BLOCK NESTED LOOP JOIN

This algorithm performs fewer disk accesses.

→ For every block in **R**, it scans **S** once

Cost: $M + (M \cdot N)$

M pages
 m tuples

R(id, name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

N pages
 n tuples

S(id, value, cdate)

id	value	cddate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

BLOCK NESTED LOOP JOIN

Which one should be the outer table?

→ The smaller table in terms of # of pages

M pages
m tuples

R(id, name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

N pages
n tuples

S(id, value, cdate)

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

BLOCK NESTED LOOP JOIN

Example database:

→ Table **R**: $M = 1000$, $m = 100,000$

→ Table **S**: $N = 500$, $n = 40,000$

Cost Analysis:

→ $M + (M \cdot N) = 1000 + (1000 \cdot 500) = 501,000$ IOs

→ At 0.1 ms/IO, Total time ≈ 50 seconds

BLOCK NESTED LOOP JOIN

What if we have **B** buffers available?

- Use **B-2** buffers for scanning the outer table.
- Use one buffer for the inner table, one buffer for storing output.

R(id, name)

M pages
m tuples

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

S(id, value, cdate)

N pages
n tuples

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

BLOCK NESTED LOOP JOIN

```

foreach  $B-2$  blocks  $b_R \in R$ :
  foreach block  $b_S \in S$ :
    foreach tuple  $r \in b_R$ :
      foreach tuple  $s \in b_S$ :
        emit, if  $r$  and  $s$  match
  
```

$R(id, name)$

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

M pages
 m tuples

$S(id, value, cdate)$

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

N pages
 n tuples

BLOCK NESTED LOOP JOIN

This algorithm uses $B-2$ buffers for scanning R .

Cost: $M + \lceil M / (B-2) \rceil \cdot N$

What if the outer relation completely fits in memory ($B > M+2$)?

→ **Cost:** $M + N = 1000 + 500 = 1500$ IOs

→ At 0.1ms/IO, Total time ≈ 0.15 seconds

NESTED LOOP JOIN

Why do basic nested loop joins suck?

→ For each tuple in the outer table, we must do a sequential scan to check for a match in the inner table.

We can avoid sequential scans by using an index to find inner table matches.

→ Use an existing index for the join.

→ Build one on the fly (hash table, B+Tree).

INDEX NESTED LOOP JOIN

```

foreach tuple  $r \in R$ :
  foreach tuple  $s \in \text{Index}(r_i = s_j)$ :
    emit, if  $r$  and  $s$  match
  
```

M pages
 m tuples

$R(\text{id}, \text{name})$

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

$S(\text{id}, \text{value}, \text{cdate})$

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

$\text{Index}(S.\text{id})$



N pages
 n tuples

INDEX NESTED LOOP JOIN

Assume the cost of each index probe is some constant **C** per tuple.

Cost: $M + (m \cdot C)$

M pages
 m tuples

R(id, name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
400	Raekwon

S(id, value, cdate)

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

Index(S.id)



N pages
 n tuples

NESTED LOOP JOIN

Pick the smaller table as the outer table.

Buffer as much of the outer table in memory as possible.

Loop over the inner table or use an index.



SORT-MERGE JOIN

Phase #1: Sort

- Sort both tables on the join key(s).
- We can use the external merge sort algorithm that we talked about last class.

Phase #2: Merge

- Step through the two sorted tables with cursors and emit matching tuples.
- May need to backtrack depending on the join type.

SORT-MERGE JOIN

```
sort R, S on join keys
cursorR ← Rsorted, cursorS ← Ssorted
while cursorR and cursorS:
    if cursorR > cursorS:
        increment cursorS
    if cursorR < cursorS:
        increment cursorR
    elif cursorR and cursorS match:
        emit
        increment cursorS
```

SORT-MERGE JOIN

R(id,name)

id	name
600	MethodMan
200	GZA
100	Andy
300	ODB
500	RZA
700	Ghostface
200	GZA
400	Raekwon

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
500	7777	10/2/2019
400	6666	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```


SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

↑
Sort!

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

↑
Sort!

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface



S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)



id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)



id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface



S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface



S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019
500	RZA	500	7777	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface



S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019
500	RZA	500	7777	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface

S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019

```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019
500	RZA	500	7777	10/2/2019

SORT-MERGE JOIN

R(id,name)

id	name
100	Andy
200	GZA
200	GZA
300	ODB
400	Raekwon
500	RZA
600	MethodMan
700	Ghostface



S(id,value,cdate)

id	value	cdate
100	2222	10/2/2019
100	9999	10/2/2019
200	8888	10/2/2019
400	6666	10/2/2019
500	7777	10/2/2019



```
SELECT R.id, S.cdate
FROM R JOIN S
      ON R.id = S.id
WHERE S.value > 100
```

Output Buffer

R.id	R.name	S.id	S.value	S.cdate
100	Andy	100	2222	10/2/2019
100	Andy	100	9999	10/2/2019
200	GZA	200	8888	10/2/2019
200	GZA	200	8888	10/2/2019
400	Raekwon	200	6666	10/2/2019
500	RZA	500	7777	10/2/2019

SORT-MERGE JOIN

Sort Cost (**R**): $2M \cdot (1 + \lceil \log_{B-1} [M / B] \rceil)$

Sort Cost (**S**): $2N \cdot (1 + \lceil \log_{B-1} [N / B] \rceil)$

Merge Cost: $(M + N)$

Total Cost: Sort + Merge



SORT-MERGE JOIN

Example database:

→ Table **R**: $M = 1000$, $m = 100,000$

→ Table **S**: $N = 500$, $n = 40,000$

With $B=100$ buffer pages, both **R** and **S** can be sorted in two passes:

→ Sort Cost (**R**) = $2000 \cdot (1 + \lceil \log_{99} 1000 / 100 \rceil) = 4000$ IOs

→ Sort Cost (**S**) = $1000 \cdot (1 + \lceil \log_{99} 500 / 100 \rceil) = 2000$ IOs

→ Merge Cost = $(1000 + 500) = 1500$ IOs

→ Total Cost = $4000 + 2000 + 1500 = 7500$ IOs

→ At 0.1 ms/IO, Total time ≈ 0.75 seconds

SORT-MERGE JOIN

The worst case for the merging phase is when the join attribute of all of the tuples in both relations contain the same value.

Cost: $(M \cdot N) + (\text{sort cost})$



WHEN IS SORT-MERGE JOIN USEFUL?

One or both tables are already sorted on join key.

Output must be sorted on join key.

The input relations may be sorted by either by an explicit sort operator, or by scanning the relation using an index on the join key.

HASH JOIN

If tuple $\mathbf{r} \in \mathbf{R}$ and a tuple $\mathbf{s} \in \mathbf{S}$ satisfy the join condition, then they have the same value for the join attributes.

If that value is hashed to some partition $\mathbf{i}$, the $\mathbf{R}$ tuple must be in $\mathbf{r}_i$ and the $\mathbf{S}$ tuple in $\mathbf{s}_i$.

Therefore, $\mathbf{R}$ tuples in $\mathbf{r}_i$ need only to be compared with $\mathbf{S}$ tuples in $\mathbf{s}_i$.

BASIC HASH JOIN ALGORITHM

Phase #1: Build

- Scan the outer relation and populate a hash table using the hash function h_1 on the join attributes.

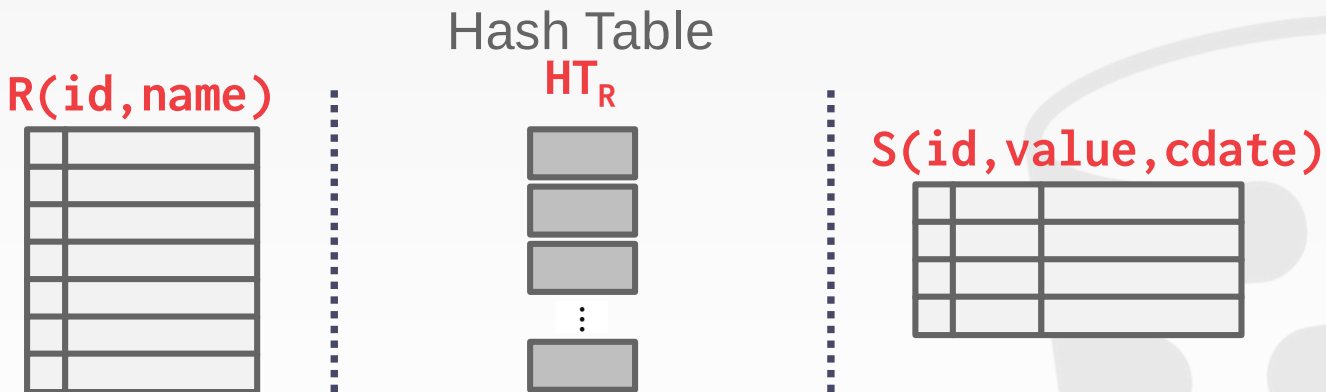
Phase #2: Probe

- Scan the inner relation and use h_1 on each tuple to jump to a location in the hash table and find a matching tuple.

BASIC HASH JOIN ALGORITHM

```

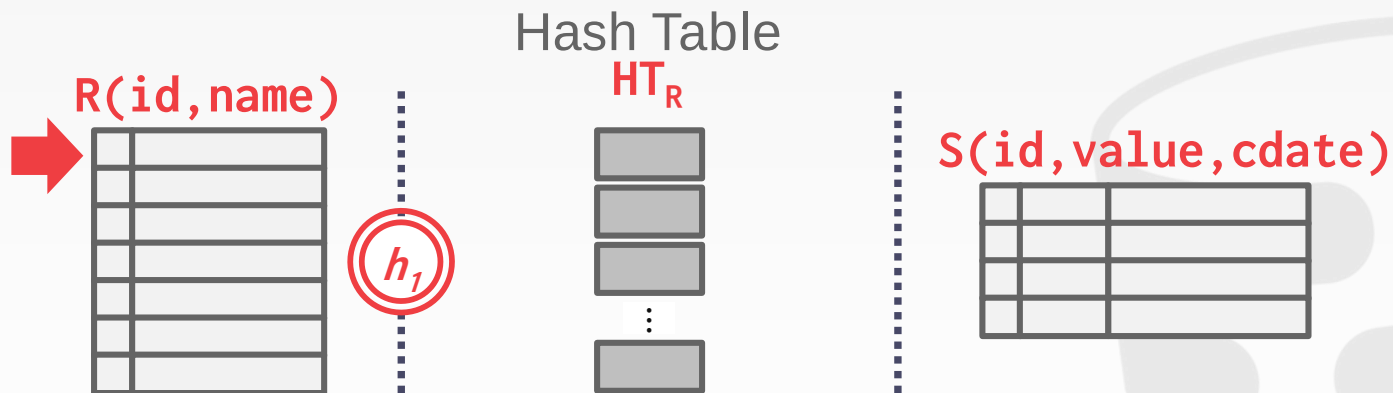
build hash table  $HT_R$  for  $R$ 
foreach tuple  $s \in S$ 
  output, if  $h_1(s) \in HT_R$ 
  
```



BASIC HASH JOIN ALGORITHM

```

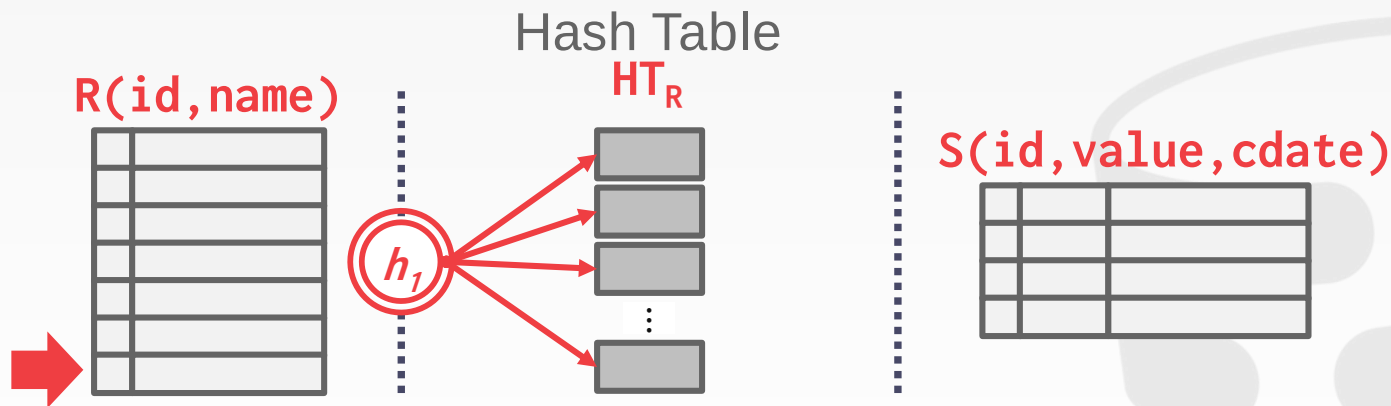
build hash table  $HT_R$  for  $R$ 
foreach tuple  $s \in S$ 
  output, if  $h_1(s) \in HT_R$ 
  
```



BASIC HASH JOIN ALGORITHM

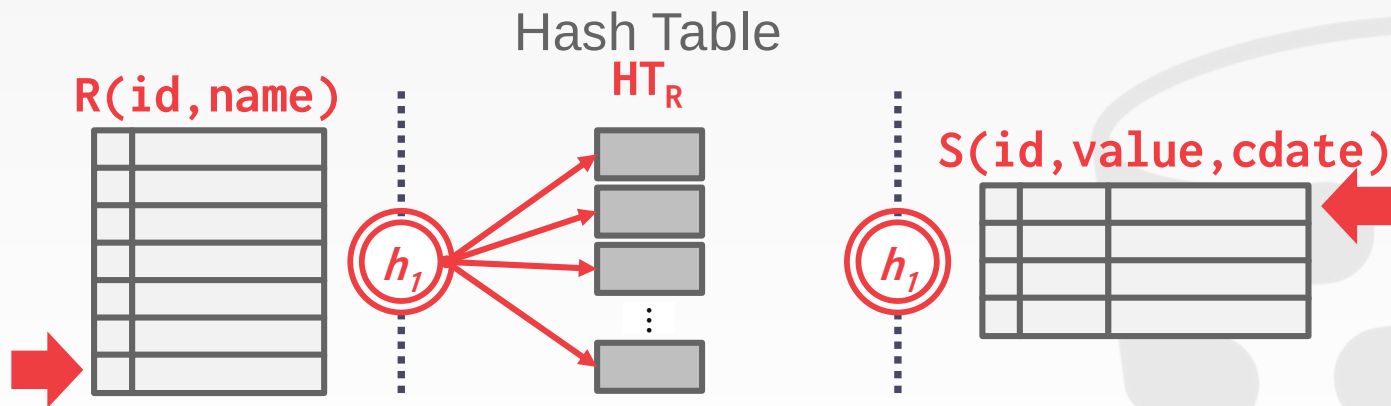
```

build hash table  $HT_R$  for  $R$ 
foreach tuple  $s \in S$ 
  output, if  $h_1(s) \in HT_R$ 
  
```



BASIC HASH JOIN ALGORITHM

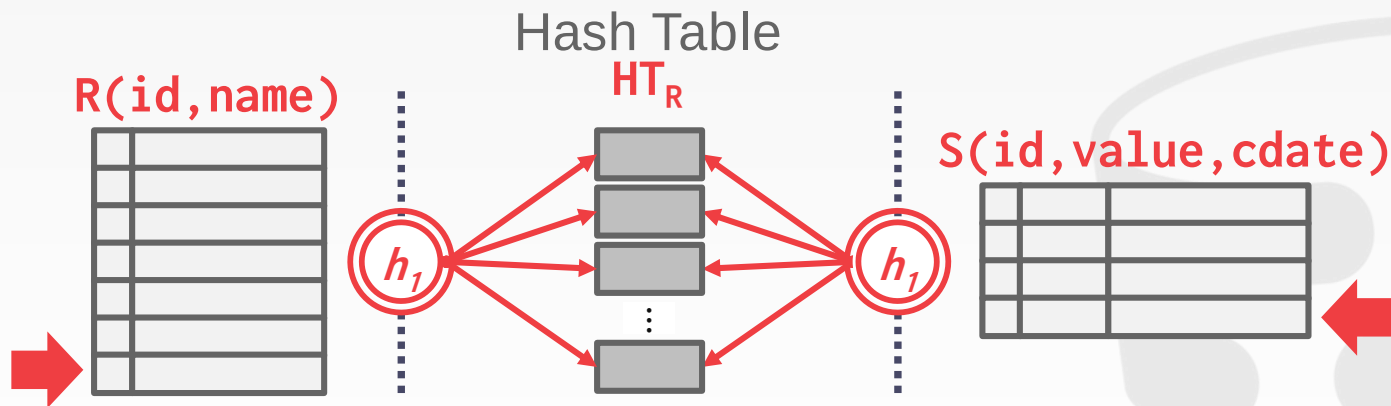
```
build hash table  $HT_R$  for  $R$   
foreach tuple  $s \in S$   
  output, if  $h_1(s) \in HT_R$ 
```



BASIC HASH JOIN ALGORITHM

```

build hash table  $HT_R$  for  $R$ 
foreach tuple  $s \in S$ 
  output, if  $h_1(s) \in HT_R$ 
  
```



HASH TABLE CONTENTS

Key: The attribute(s) that the query is joining the tables on.

Value: Varies per implementation.

→ Depends on what the operators above the join in the query plan expect as its input.

HASH TABLE VALUES

Approach #1: Full Tuple

- Avoid having to retrieve the outer relation's tuple contents on a match.
- Takes up more space in memory.

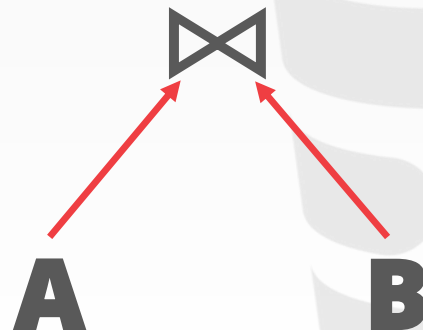
Approach #2: Tuple Identifier

- Ideal for column stores because the DBMS doesn't fetch data from disk it doesn't need.
- Also better if join selectivity is low.

PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

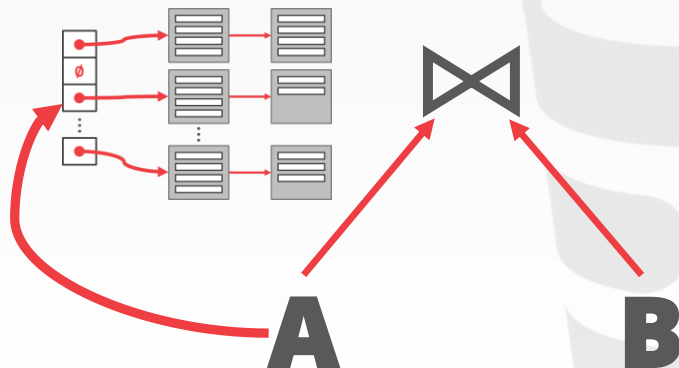
- Threads check the filter before probing the hash table.
This will be faster since the filter will fit in CPU caches.
- Sometimes called *sideways information passing*.



PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

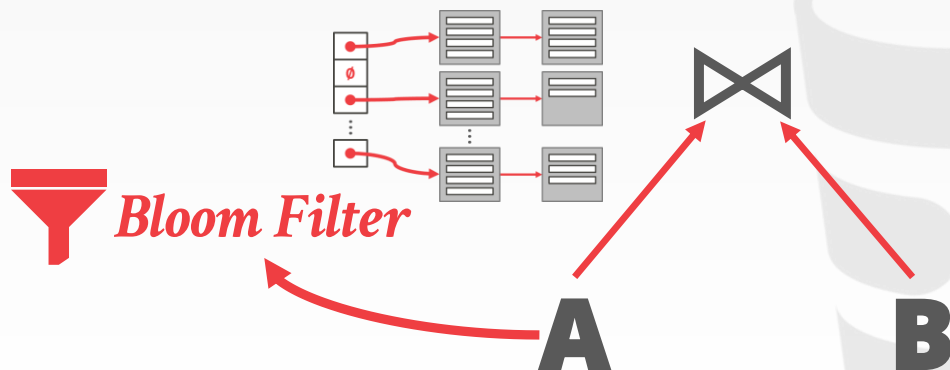
- Threads check the filter before probing the hash table.
This will be faster since the filter will fit in CPU caches.
- Sometimes called *sideways information passing*.



PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

- Threads check the filter before probing the hash table.
This will be faster since the filter will fit in CPU caches.
- Sometimes called *sideways information passing*.



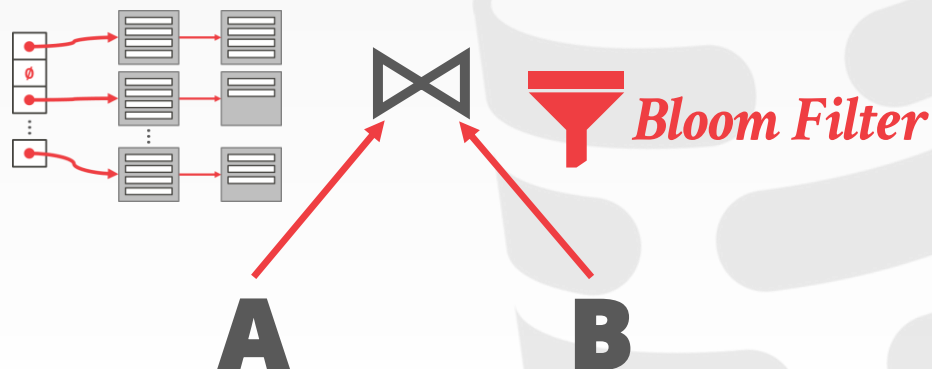
PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

→ Threads check the filter before probing the hash table.

This will be faster since the filter will fit in CPU caches.

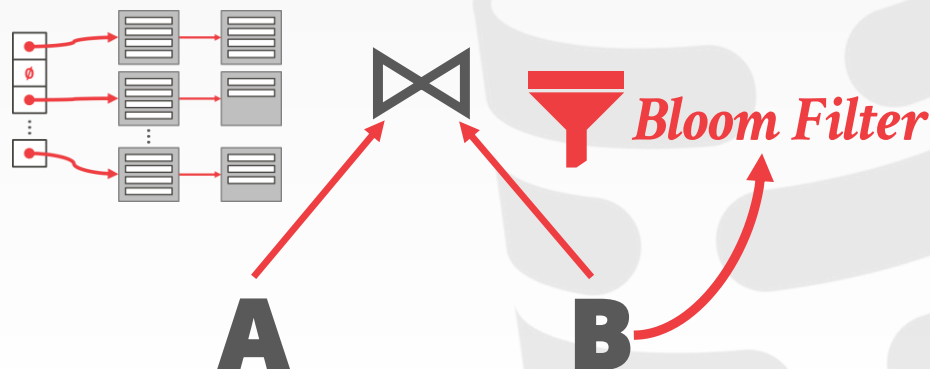
→ Sometimes called *sideways information passing*.



PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

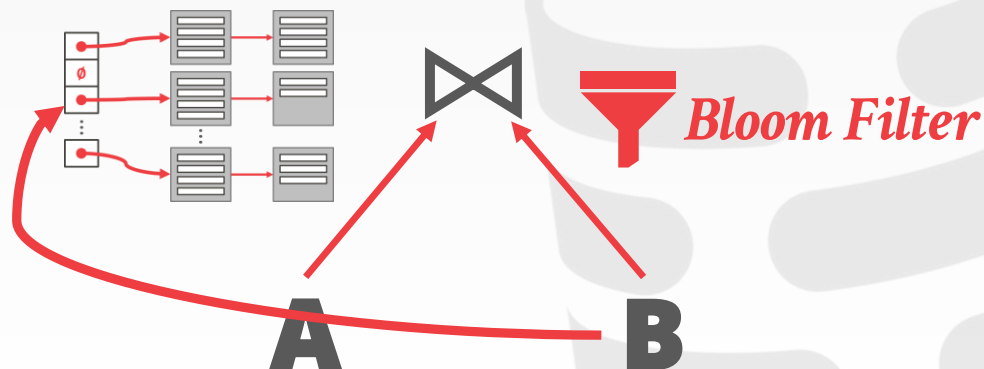
- Threads check the filter before probing the hash table.
This will be faster since the filter will fit in CPU caches.
- Sometimes called *sideways information passing*.



PROBE PHASE OPTIMIZATION

Create a Bloom Filter during the build phase when the key is likely to not exist in the hash table.

- Threads check the filter before probing the hash table.
This will be faster since the filter will fit in CPU caches.
- Sometimes called *sideways information passing*.



HASH JOIN

What happens if we do not have enough memory to fit the entire hash table?

We do not want to let the buffer pool manager swap out the hash table pages at a random.

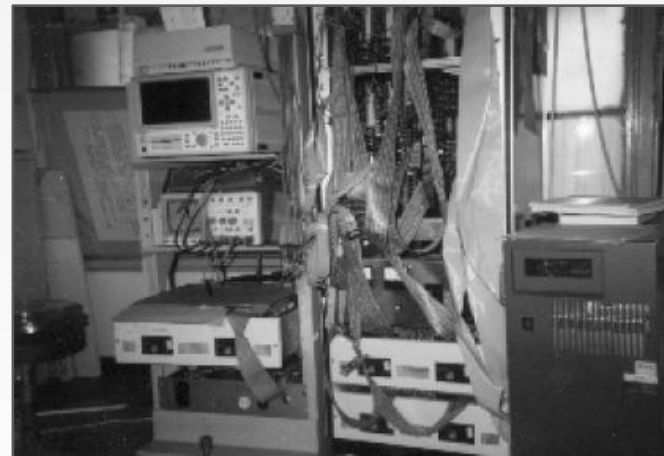


GRACE HASH JOIN

Hash join when tables do not fit in memory.

- **Build Phase:** Hash both tables on the join attribute into partitions.
- **Probe Phase:** Compares tuples in corresponding partitions for each table.

Named after the GRACE database machine from Japan in the 1980s.



GRACE
University of Tokyo

HASH JOIN

IBM DB2 Analytics Accelerator - GSE Management Summit

Choosing the best fit Key indicators

IBM Netezza

- Performance and Price/performance leader
- Speed and ease of deployment and administration

IBM Netezza standalone appliance



CLUSTRIX APPLIANCE

Clustrix Appliance 3 Node Cluster (CLX 4110)

- 24 Intel Xeon CPU cores
- 144GB RAM
- 6GB NVRAM
- 1.35TB Intel SSD
 - (2.7TB raw)
- Low-latency I/O



Complete Family Of Database Machines For OLTP, Data Warehousing & Consolidated Workloads

Oracle Exadata X2-2



- Quarter, Half, Full and Multi-Racks

Oracle Exadata X2-8



- Full and Multi-Racks

ORACLE

Named after the GRACE
machine from Japan



GRACE
University of Tokyo

IBM DB2 Analytics Accelerator - GSE Management Summit

Choosing the best fit

Key indicators

IBM Netezza

- Performance and Price/performance leader
- Speed and ease of deployment and administration

IBM Netezza standalone appliance



CLUSTRIX APPLIANCE

Clustrix Appliance 3 Node Cluster (CLX 4110)

- 24 Intel Xeon CPU cores
- 144GB RAM
- 6GB NVRAM
- 1.35TB Intel SSD (2.7TB raw)
- Low-latency I/O



Complete Family Of For OLTP, Data Warehousing

Oracle Exadata X2-2



- Quarter, Half, Full and Multi-Racks



- Full and Multi-Racks

ORACLE

HASH JOIN

Yellowbrick Data Warehouse Architecture

Real-time Feeds
Ingest IoT or OLTP data
Capture 100,000s
of rows per second



Periodic Bulk Loads
Capture terabytes
of data, petabytes
over time



Load and Transform
Use existing ETL tools including
intensive push-down ELT



Interactive Applications
Serve short queries in
under 100 milliseconds



Powerful Analytics
Respond to
complex BI queries
in just a few seconds



Business Critical Reporting
Workload management
for prioritized responses



Source: yellowbrickdata.com

Named after the GRACE
machine from Japan

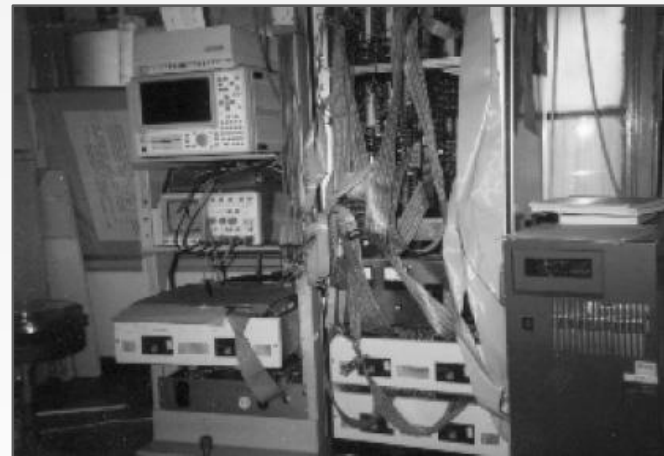
GRACE
University of Tokyo

GRACE HASH JOIN

Hash join when tables do not fit in memory.

- **Build Phase:** Hash both tables on the join attribute into partitions.
- **Probe Phase:** Compares tuples in corresponding partitions for each table.

Named after the GRACE database machine from Japan in the 1980s.

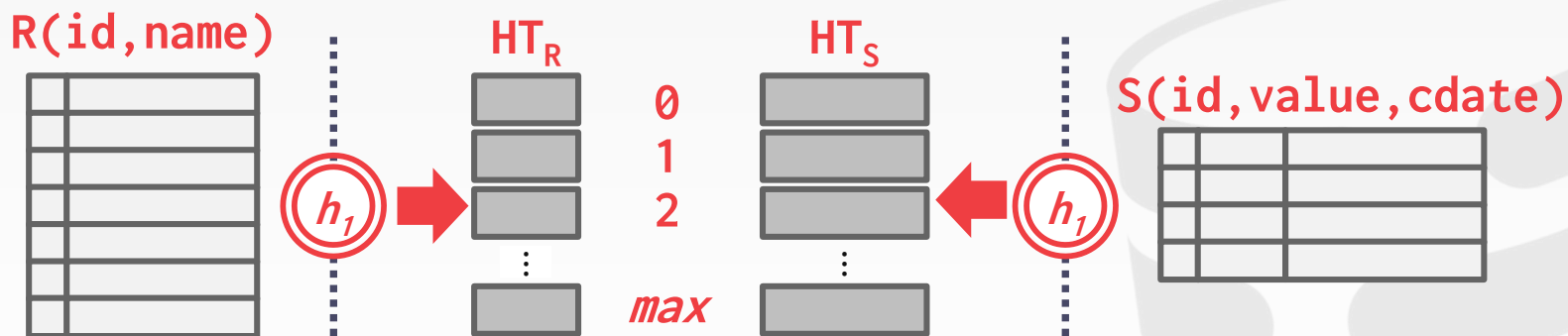


GRACE
University of Tokyo

GRACE HASH JOIN

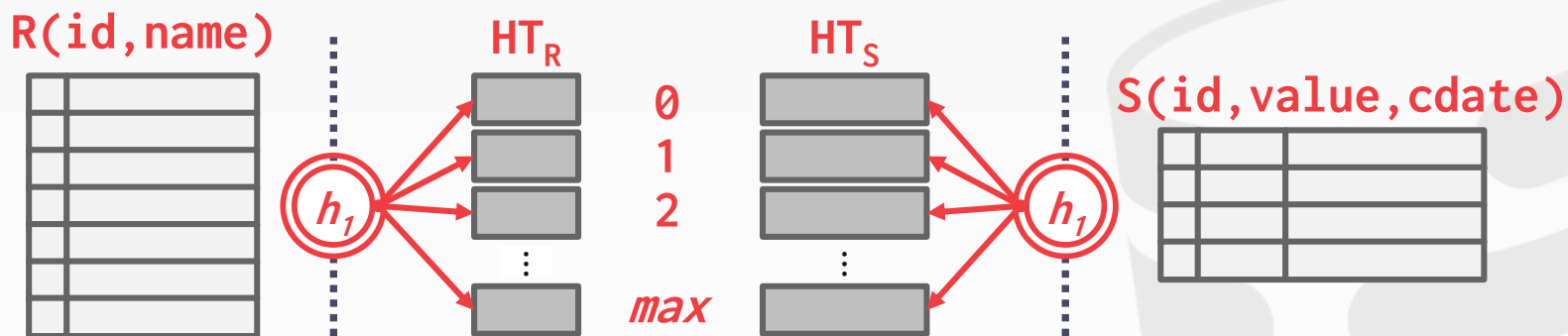
Hash **R** into $(0, 1, \dots, \text{max})$ buckets.

Hash **S** into the same # of buckets with the same hash function.



GRACE HASH JOIN

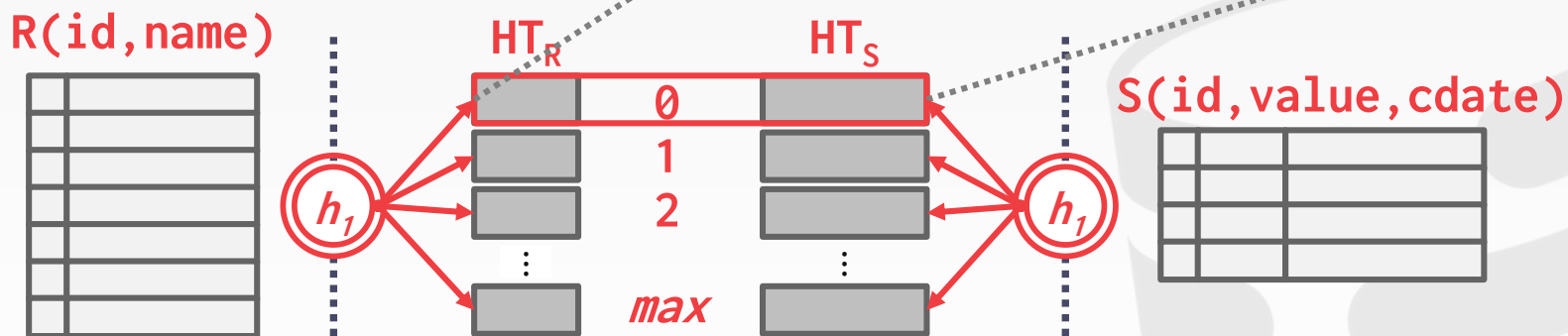
Join each pair of matching buckets
between **R** and **S**.



GRACE HASH JOIN

Join each pair of matching buckets between **R** and **S**.

```
foreach tuple r ∈ bucketR,0:
  foreach tuple s ∈ bucketS,0:
    emit, if match(r, s)
```



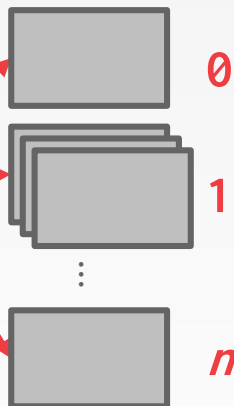
GRACE HASH JOIN

If the buckets do not fit in memory, then use **recursive partitioning** to split the tables into chunks that will fit.

- Build another hash table for **bucket_{R,i}** using hash function **h_2** (with **$h_2 \neq h_1$**).
- Then probe it for each tuple of the other table's bucket at that level.

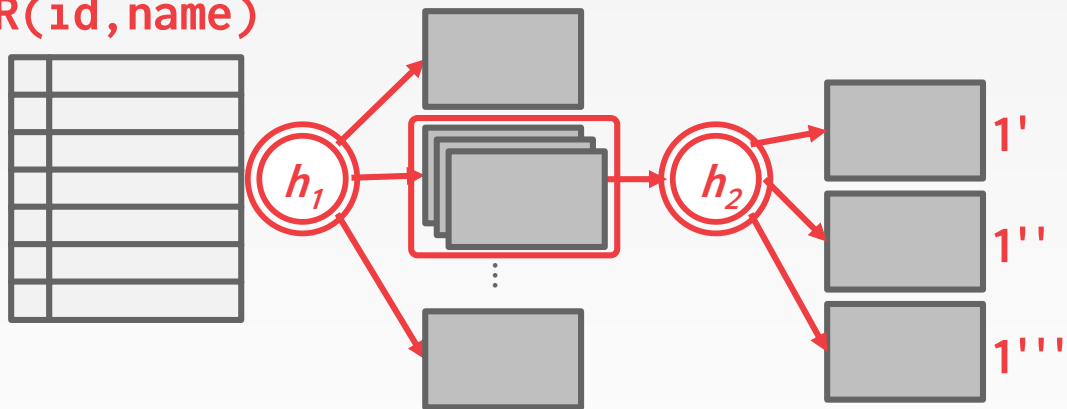
RECURSIVE PARTITIONING

$R(id, name)$

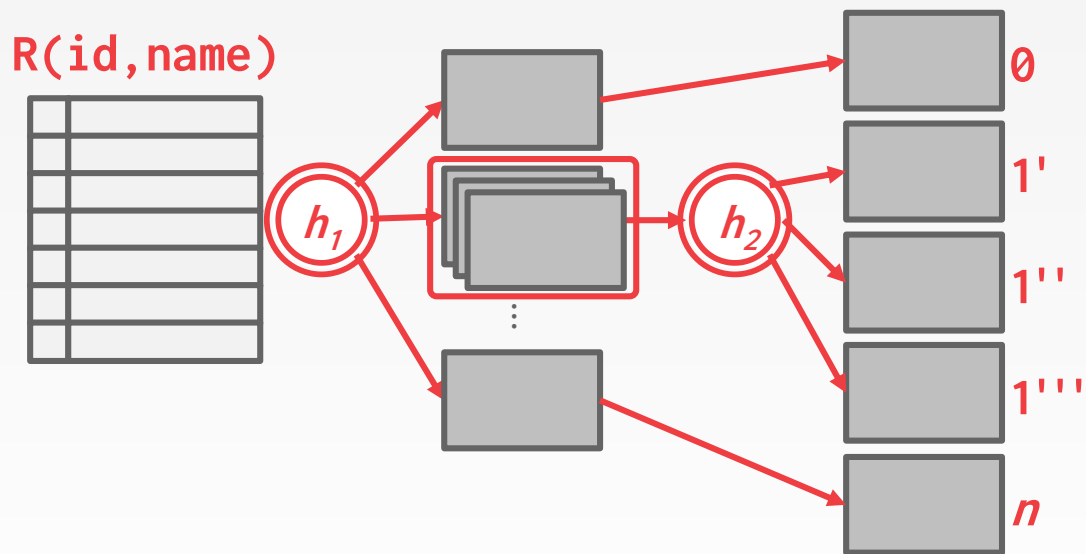


RECURSIVE PARTITIONING

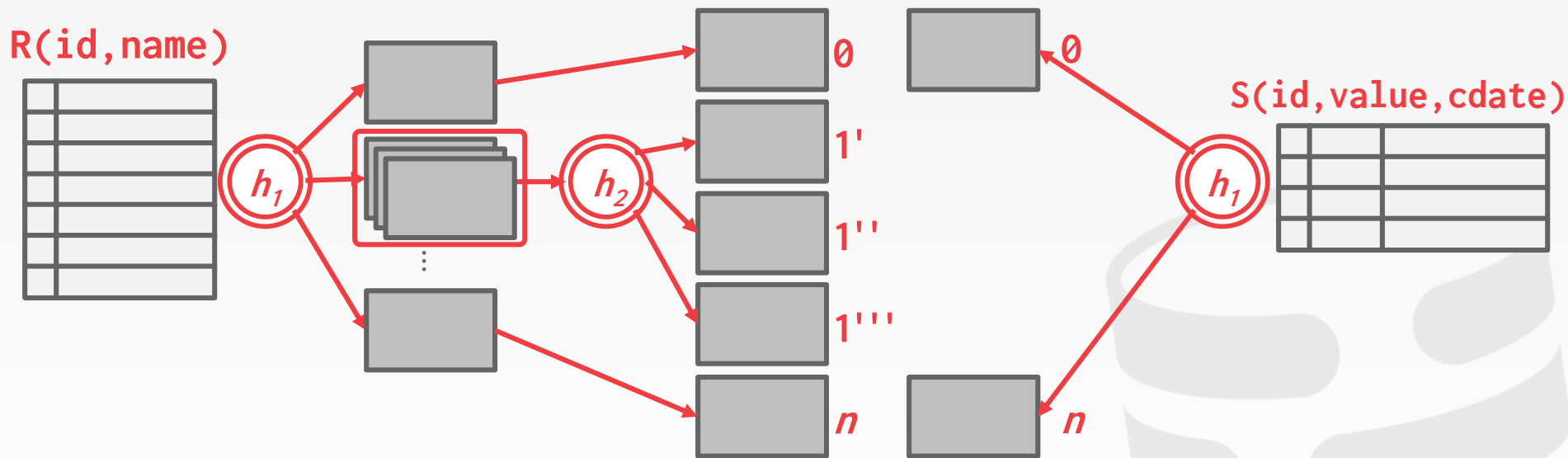
$R(id, name)$



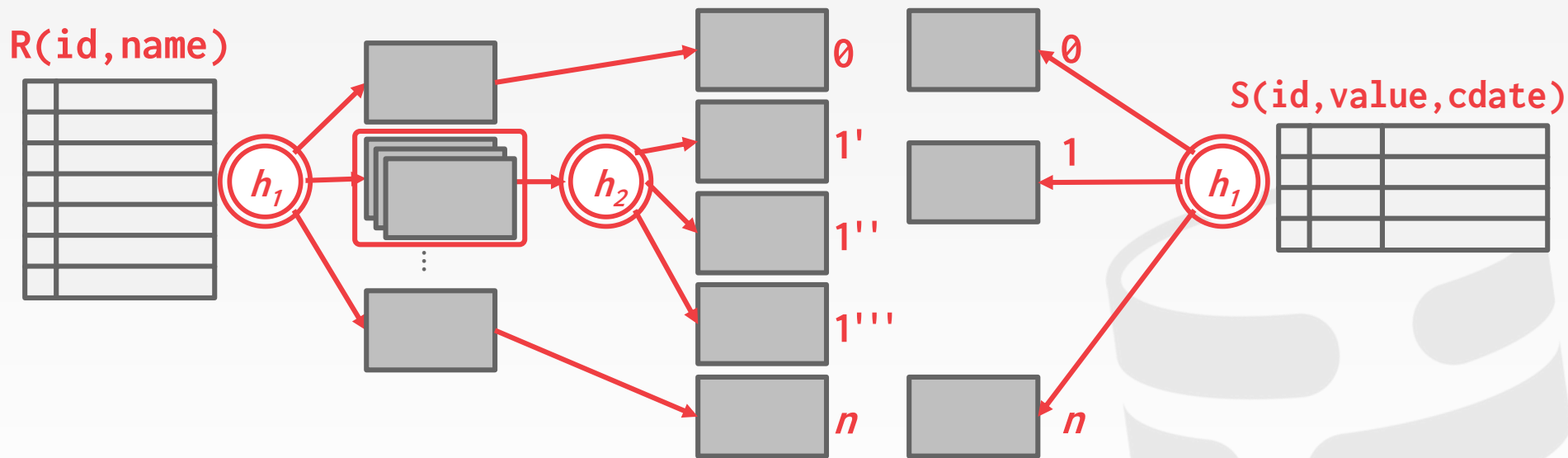
RECURSIVE PARTITIONING



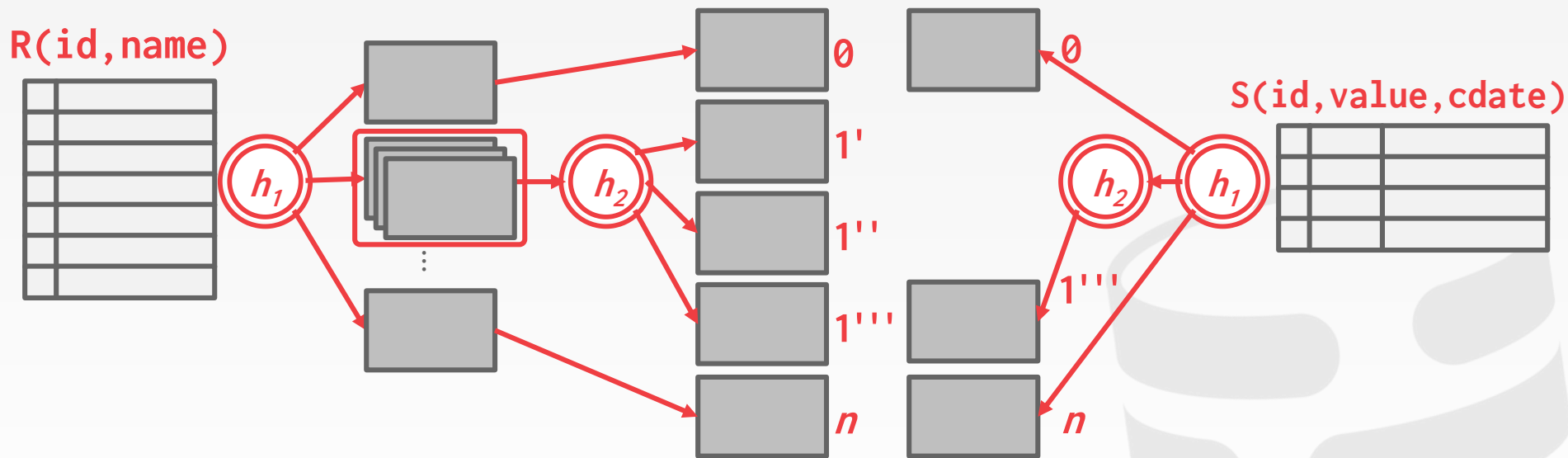
RECURSIVE PARTITIONING



RECURSIVE PARTITIONING



RECURSIVE PARTITIONING



GRACE HASH JOIN

Cost of hash join?

- Assume that we have enough buffers.
- Cost: $3(M + N)$

Partitioning Phase:

- Read+Write both tables
- $2(M+N)$ IOs

Probing Phase:

- Read both tables
- $M+N$ IOs



GRACE HASH JOIN

Example database:

→ $M = 1000$, $m = 100,000$

→ $N = 500$, $n = 40,000$

Cost Analysis:

→ $3 \cdot (M + N) = 3 \cdot (1000 + 500) = 4,500$ IOs

→ At 0.1 ms/IO, Total time ≈ 0.45 seconds



OBSERVATION

If the DBMS knows the size of the outer table, then it can use a static hash table.

→ Less computational overhead for build / probe operations.

If we do not know the size, then we have to use a dynamic hash table or allow for overflow pages.

JOIN ALGORITHMS: SUMMARY

Algorithm	IO Cost	Example
Simple Nested Loop Join	$M + (m \cdot N)$	1.3 hours
Block Nested Loop Join	$M + (M \cdot N)$	50 seconds
Index Nested Loop Join	$M + (M \cdot \log N)$	20 seconds
Sort-Merge Join	$M + N + (\text{sort cost})$	0.59 seconds
Hash Join	$3(M + N)$	0.45 seconds

CONCLUSION

Hashing is almost always better than sorting for operator execution.

Caveats:

- Sorting is better on non-uniform data.
- Sorting is better when result needs to be sorted.

Good DBMSs use either or both.

NEXT CLASS

Composing operators together to execute queries.



CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

数据库

聚合

内存管理

树索引

排序

查询规划

索引构建

分组

并发控制

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2：扫码点击**底部菜单栏**

称为 **AI 内容创作者**？回复 [添砖加瓦]